

# Introducción teórico-práctica al Aprendizaje por Refuerzo

Clase 6— TD learning || SARSA, Q-learning

Profesor: Matias Bilkis



**UBA**  
Universidad de Buenos Aires



# Repaso clase 5

Stanisław Ulam



# On-policy

On-policy first-visit MC control (for  $\varepsilon$ -soft policies), estimates  $\pi \approx \pi_*$

Algorithm parameter: small  $\varepsilon > 0$

Initialize:

$\pi \leftarrow$  an arbitrary  $\varepsilon$ -soft policy

$Q(s, a) \in \mathbb{R}$  (arbitrarily), for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$  empty list, for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following  $\pi$ :  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode,  $t = T-1, T-2, \dots, 0$ :

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair  $S_t, A_t$  appears in  $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$ :

Append  $G$  to  $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \operatorname{argmax}_a Q(S_t, a)$  (with ties broken arbitrarily)

For all  $a \in \mathcal{A}(S_t)$ :

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

# Off policy

Off-policy MC control, for estimating  $\pi \approx \pi_*$

Initialize, for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ :

$Q(s, a) \in \mathbb{R}$  (arbitrarily)

$C(s, a) \leftarrow 0$

$\pi(s) \leftarrow \operatorname{argmax}_a Q(s, a)$  (with ties broken consistently)

Loop forever (for each episode):

$b \leftarrow$  any soft policy

Generate an episode using  $b$ :  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

$W \leftarrow 1$

Loop for each step of episode,  $t = T-1, T-2, \dots, 0$ :

$G \leftarrow \gamma G + R_{t+1}$

$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$  (with ties broken consistently)

If  $A_t \neq \pi(S_t)$  then exit inner Loop (proceed to next episode)

$W \leftarrow W \frac{1}{b(A_t|S_t)}$

En MC tenemos que generar la trayectoria, terminarla, y aprender después.

# Repasito de bandits...

Concepto de “target”



$$\hat{q}_{n+1}(a) = \hat{q}_n(a) + \frac{1}{n} \left( R_n - \hat{q}_n(a) \right)$$

$$\hat{q}_{\text{new}} = \hat{q}_{\text{old}} + \alpha_{\text{step}} [R_{\text{experience}} - \hat{q}_{\text{old}}]$$



# Temporal-difference learning



# Motivaciones

Aprender [actualizar] dentro del episodio.

Utilizar los estimadores de  $V(S_{t+1})$  para  $V(S_t)$ , como hacíamos en DP “**bootstrap**”

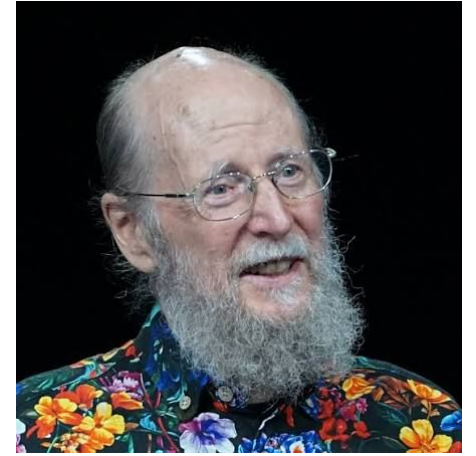
Explotar un poco más la Eq. de Bellman (que MC no la explota, más allá de estimar de manera “trivial” el  $E[G_t]$ )

$$v_{\pi}(s) = \sum_{r,a} r p(r|s,a) \pi(a|s) + \gamma \sum_{a,s'} \pi(a|s) p(s'|s,a) v_{\pi}(s')$$

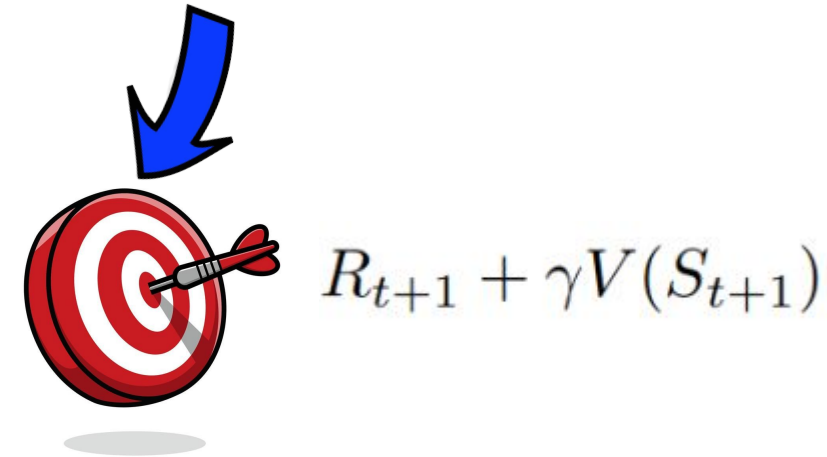
# TD(0)... ¿en qué consiste?



$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$



$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$



# TD(0)... ¿en qué consiste?

## Tabular TD(0) for estimating $v_\pi$

Input: the policy  $\pi$  to be evaluated

Algorithm parameter: step size  $\alpha \in (0, 1]$

Initialize  $V(s)$ , for all  $s \in \mathcal{S}^+$ , arbitrarily except that  $V(\text{terminal}) = 0$

Loop for each episode:

    Initialize  $S$

    Loop for each step of episode:

$A \leftarrow$  action given by  $\pi$  for  $S$

        Take action  $A$ , observe  $R, S'$

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

    until  $S$  is terminal

# No debería haber mucho misterio



$$\mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s]$$

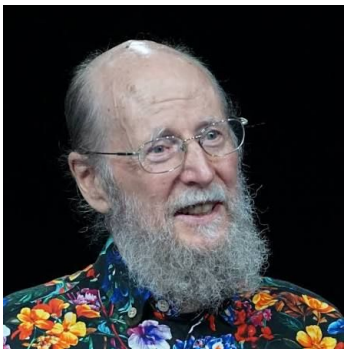
clase #4  
(Dynamic Prog)



$$\mathbb{E}_{\pi}[G_t \mid S_t = s]$$



clase #5  
(Monte Carlo)

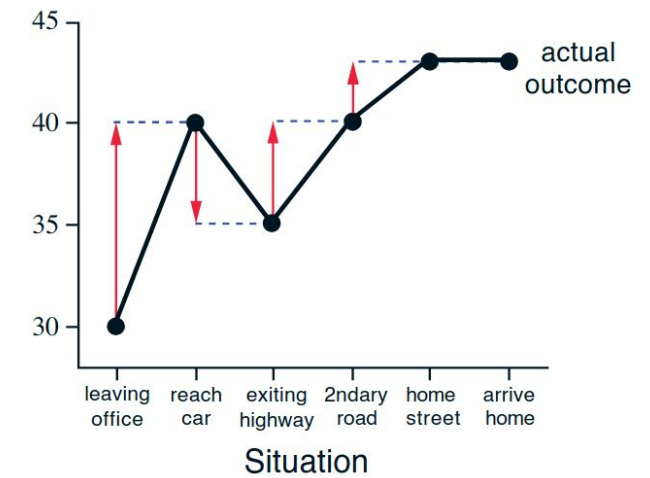
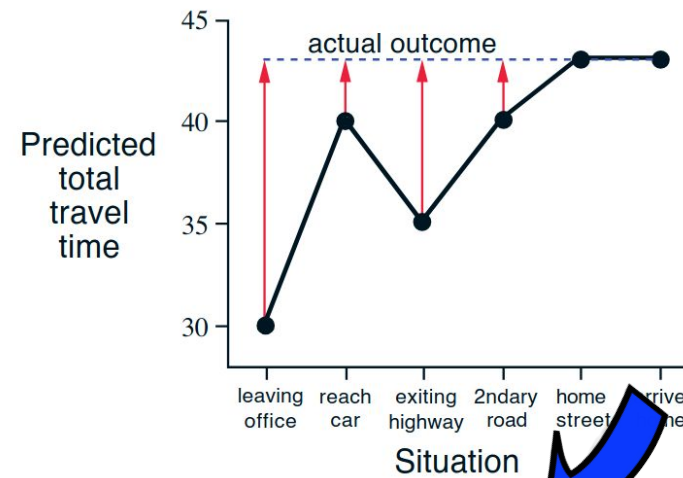



$$\mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

clase #6  
(TD)

# ¿Qué ganamos actualizando así?

Al utilizar el  $R_{t+1}$  junto con  $V(S_{t+1})$  podemos aprovechar la experiencia de  $V(S_{t+1})$  **incluso si** nuestro estimador de  $V(S_t)$  es bastante malo.



“cambiar según el  $G_t$ ”

“cambiar según el  $V(S_{t+1})$ ”

# ¿Qué ganamos actualizando así?

Al utilizar el  $R_{t+1}$  junto con  $V(S_{t+1})$  podemos aprovechar la experiencia de  $V(S_{t+1})$  **incluso si** nuestro estimador de  $V(S_t)$  es bastante malo.



Si aparece un “nuevo estado” pero que luego te lleva a estados  $S_{t+}$  que ya estimamos bien (much experiencia), TD es “mejor”.

Un ejemplo  
valor más probable  
vs.  
valor con error mínimo

# Ejemplo 6.4 hint de a qué apunta quién

Vemos el efecto de TD vs. MC en “batches”.

Tenemos un MRP (no hay acciones), y observamos 8 seqs.

A, 0, B, 0

B, 1

B, 1

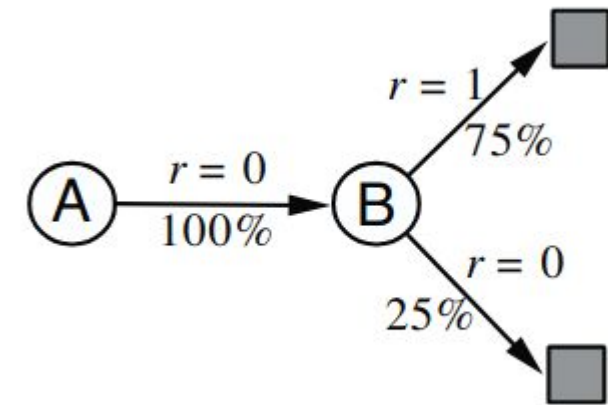
B, 1

B, 1

B, 1

B, 1

B, 0



**¿Cuanto vale  $v(B)$ ?**

**¿Cuánto vale  $V(A)$ ? → según MC vale 0**

**→ según TD vale  $V(B)$  !**

pero acá MC

no está “ni bien ni mal”

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

# Al meter el $V(S_{t+1})$ se cuele la $p(s'|sa)$

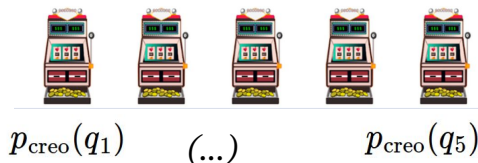
TD apunta (*pensando en batches*) al value function del MRP que mejor modela los datos, y esto viene de incluir la distribución del next-state implícita en  $V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha \left[ R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right]$$

## Thompson Sampling

Imagen (¿sirve?) gemelo digital (GD) de cada máquina / arm

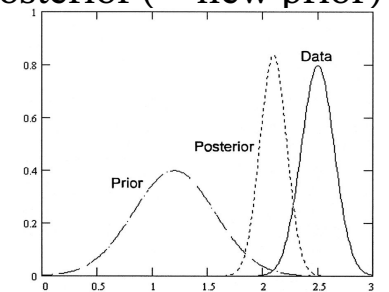
- Convertimos (“internamente”, en el GD) a  $\hat{q}(a)$  en V.A.
- Somos “greedy” en un juego interno (en el GD)  $\leftarrow$  sample All
- Actualizamos el belief usando Bayes  $p(\hat{\theta}|R_t) = \frac{p_{\text{model}}(R_t|\hat{\theta})p_{\text{prior}}(\hat{\theta})}{p_{\text{total}}(R_t)}$



## ¿Y si “modelamos” cada arm

Distribución “prior belief”  $\rightarrow$  data  $\rightarrow$  posterior (= new prior)

$$p(\hat{\theta}|R_t) = \frac{p_{\text{model}}(R_t|\hat{\theta})p_{\text{prior}}(\hat{\theta})}{p_{\text{total}}(R_t)}$$

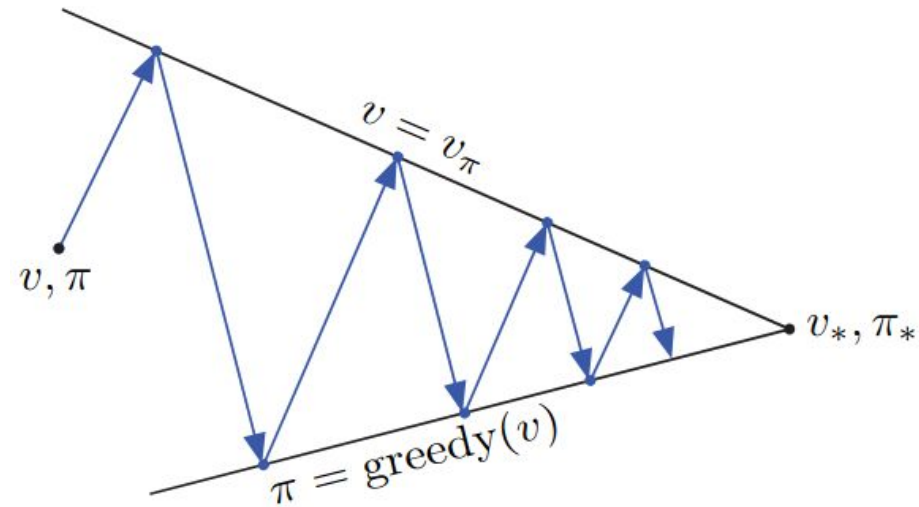
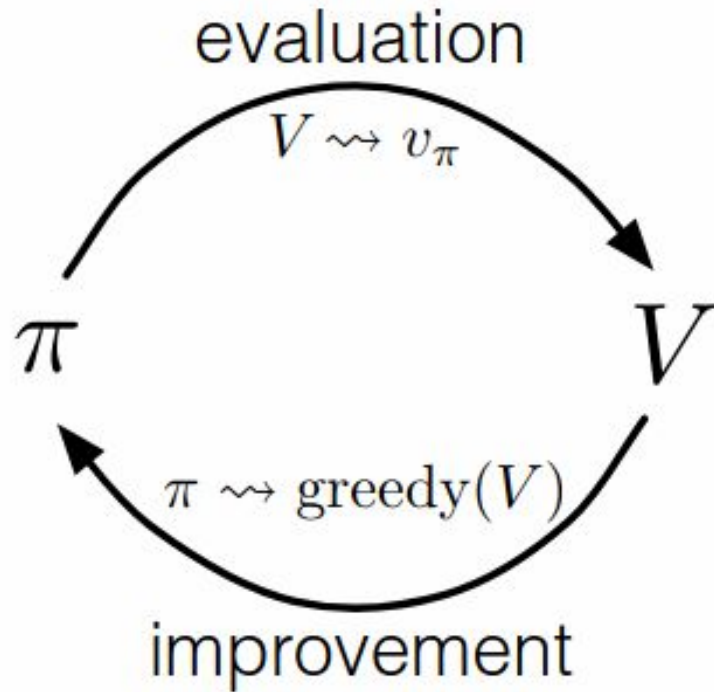


- Mantemos una creencia sobre cuán bueno es cada máquina <sup>x</sup>
- (Fijate que podría ser un parámetro del modelo!) [\[Inferencia bayesiana\]](#)
- $\rightarrow$  Mantenemos creencia (parametrizada!) del reward medio  $q$ .
- Osea, para nosotros,  $\theta \rightarrow q$  (pero esto es súper general!)

# Control

# Nada nuevo bajo el Sol, ¿no?

Otra vez, la idea es la misma que en MC, pero cambiando los update rules de las funciones de valor!



# On-policy: SARSA $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$

Habíamos visto que on-policy MC converge al mejor dentro de  $\epsilon$ -soft policies. En MC on-policy usamos:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t - Q(S_t, A_t)]$$

Si hacemos algo parecido, pero con el update de TD:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

# On-policy: SARSA $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$

On-policy first-visit MC control (for  $\varepsilon$ -soft policies), estimates  $\pi \approx \pi_*$

Algorithm parameter: small  $\varepsilon > 0$

Initialize:

$\pi \leftarrow$  an arbitrary  $\varepsilon$ -soft policy

$Q(s, a) \in \mathbb{R}$  (arbitrarily), for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$  empty list, for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following  $\pi$ :  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode,  $t = T-1, T-2, \dots, 0$ :

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair  $S_t, A_t$  appears in  $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$ :

Append  $G$  to  $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \operatorname{argmax}_a Q(S_t, a)$  (with ties broken arbitrarily)

For all  $a \in \mathcal{A}(S_t)$ :

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

Sarsa (on-policy TD control) for estimating  $Q \approx q_*$

Algorithm parameters: step size  $\alpha \in (0, 1]$ , small  $\varepsilon > 0$

Initialize  $Q(s, a)$ , for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$ , arbitrarily except that  $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize  $S$

Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)

Loop for each step of episode:

Take action  $A$ , observe  $R, S'$

Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

until  $S$  is terminal

Dif 1: update al terminar el episodio

Dif 2: el update.

Sim 1: \epsilon-greedy (on-policy!)

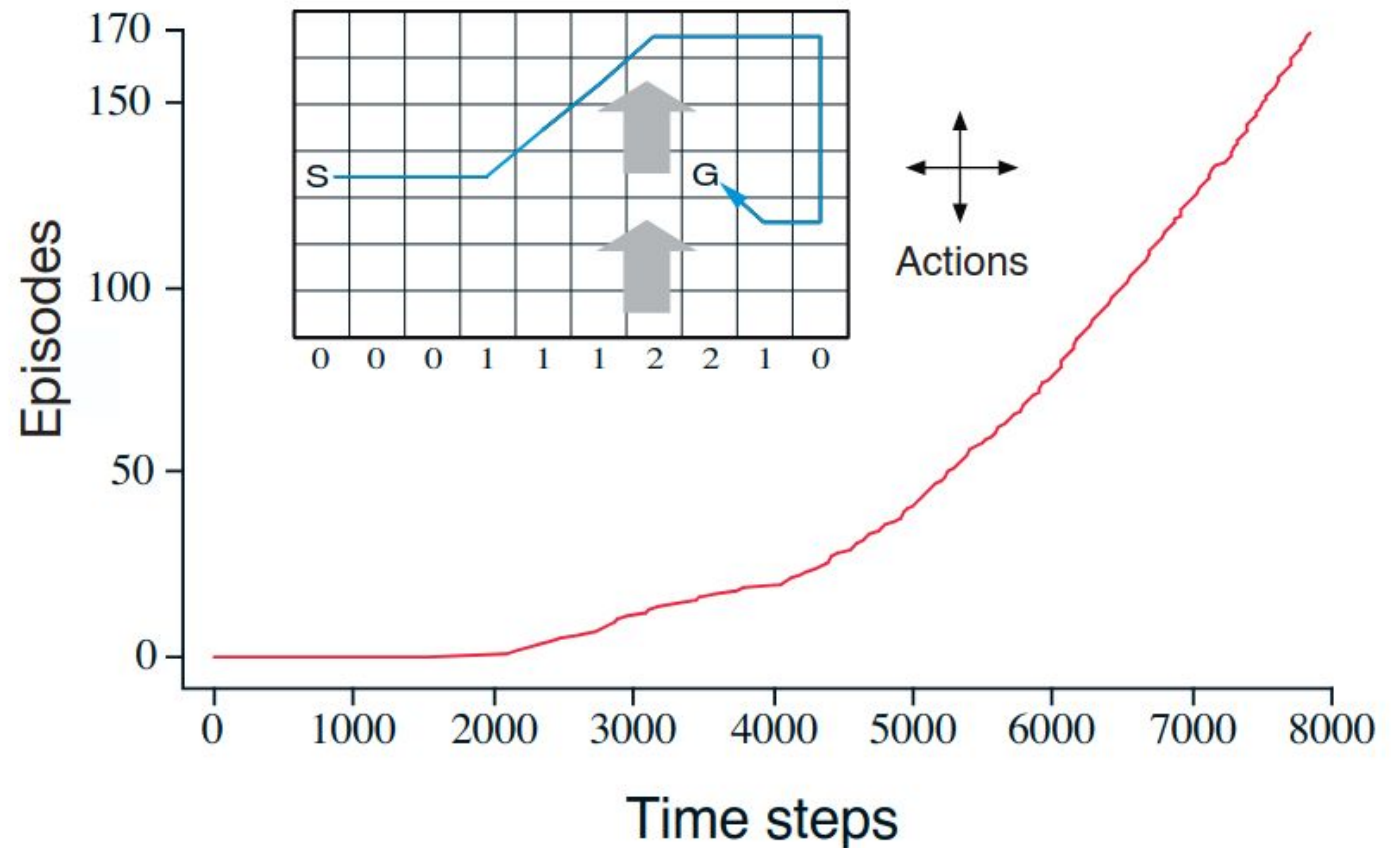
# Ej. SARSA en windy gridworld

¿Cuál sería la probabilidad de que MC encuentre el G?

$R = -1 \dots$

$\epsilon$  es importante para explorar.

Convergencia al óptimo: si exploramos todo el espacio pero a la vez decrecemos  $\epsilon$  (así  $\pi^* = \text{greedy}$ )



# On-policy: algunas reflexiones

La política que voy aprendiendo (que es la misma que voy *ejecutando*) es (en SARSA) la  $\epsilon$ -greedy

¿Qué pasa si el objetivo que ponemos es **directamente** la política óptima?

→ Necesitamos **explorar** (porque de dónde sacamos cuál es la política óptima?), y por el otro **aprender**.  
Esto nos lleva a **off-policy**

# Off-policy: Q-learning

La policy que aprendemos es la greedy;  
pero el behaviour es la  $\epsilon$ .

¿Por qué no hay **IS**?  $\rightarrow$  es solo un paso en futuro (c/ más sí)

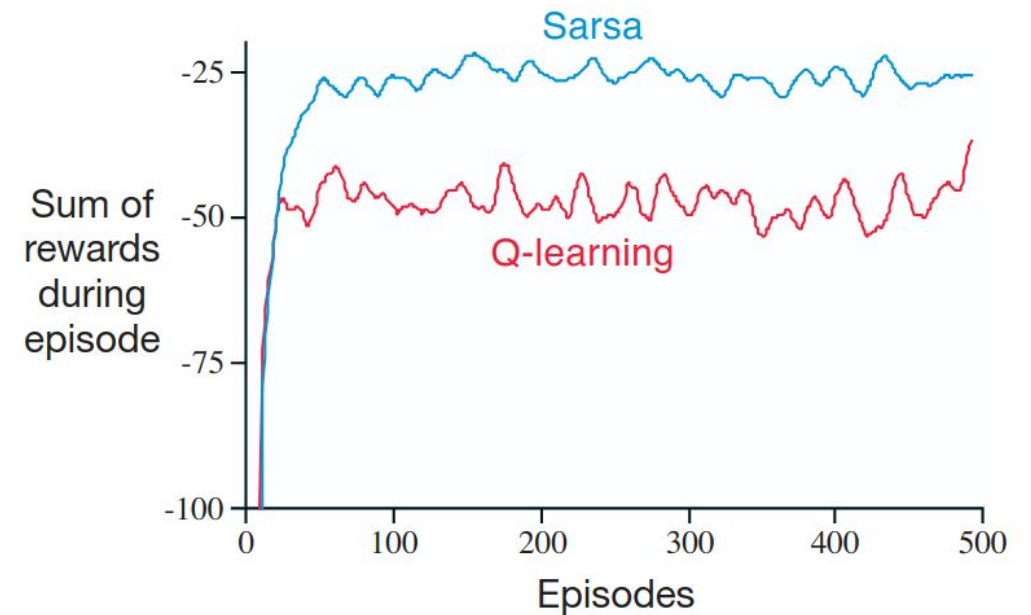
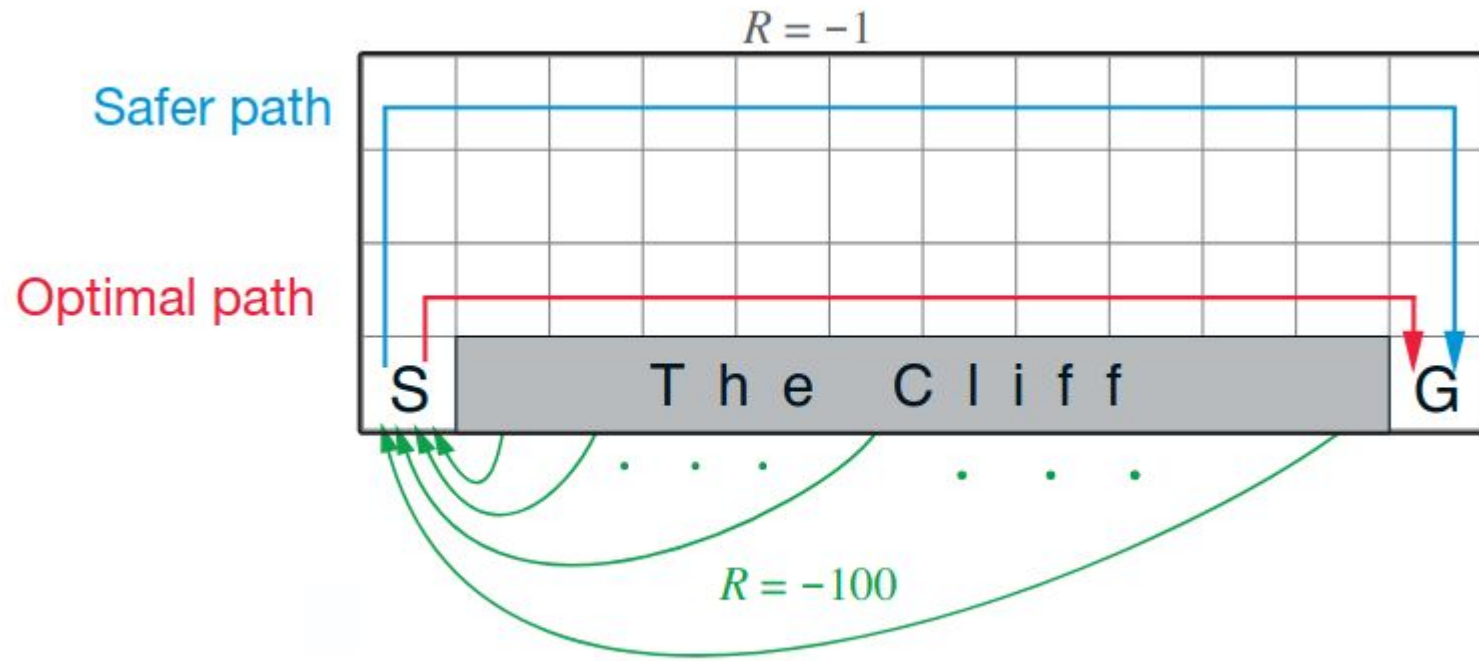
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[ R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

**La única diferencia con SARSA** es la acción del target  
(greedy vs.  $\epsilon$ -greedy)

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[ R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

# Diferencia Q-learning // SARSA

En el Cliff-walking (precipicio);  
Q-learning aprende **la política óptima** pero  
a la hora de ejecutar, le va peor... **¿why? \epsilon finito**



# Q-learning pseudocódigo

Todas las componentes de las clases anteriores...

Q-learning (off-policy TD control) for estimating  $\pi \approx \pi_*$

Algorithm parameters: step size  $\alpha \in (0, 1]$ , small  $\varepsilon > 0$

Initialize  $Q(s, a)$ , for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$ , arbitrarily except that  $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

    Initialize  $S$

    Loop for each step of episode:

        Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)

        Take action  $A$ , observe  $R, S'$

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

    until  $S$  is terminal

# Ejemplito

# Q-learning en quantum

**Quantum Physics**

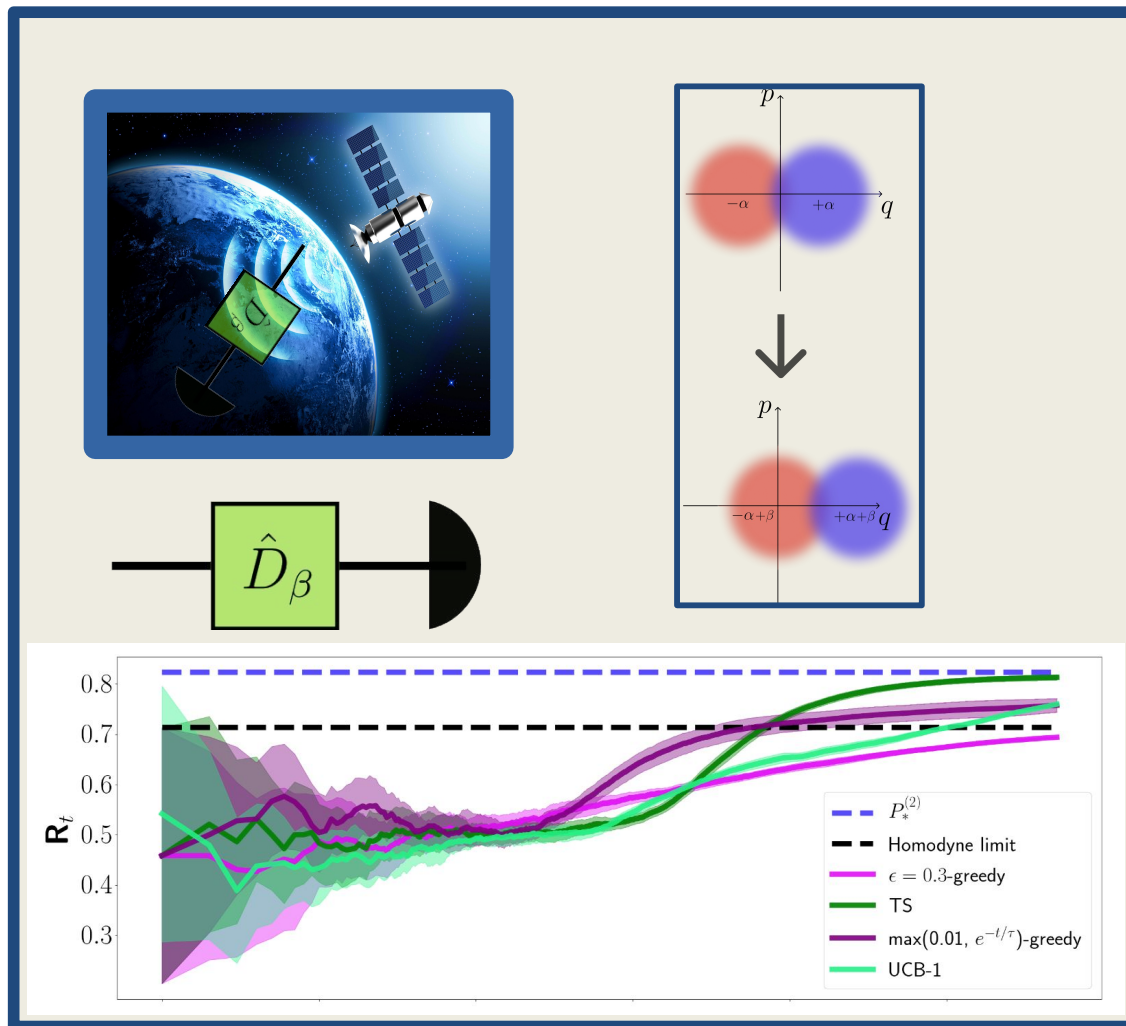
*[Submitted on 28 Jan 2020]*

**Real-time calibration of coherent-state receivers: learning by trial and error**

M. Bilkis, M. Rosati, R. Morral Yepes, J. Calsamiglia

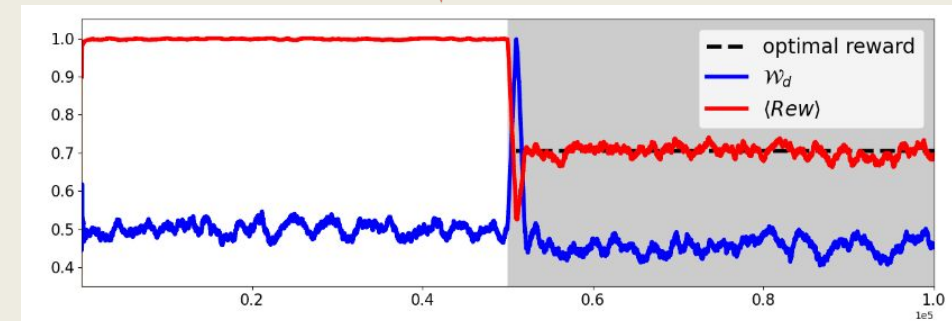
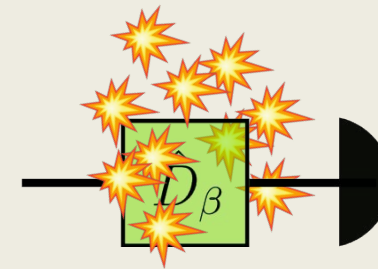
# Re-calibration of quantum devices by RL

An application to coherent-state receiver



But if the environment changes?

$$\mathcal{E}_0 \rightarrow \mathcal{E}_1$$



# Coherent-state discrimination



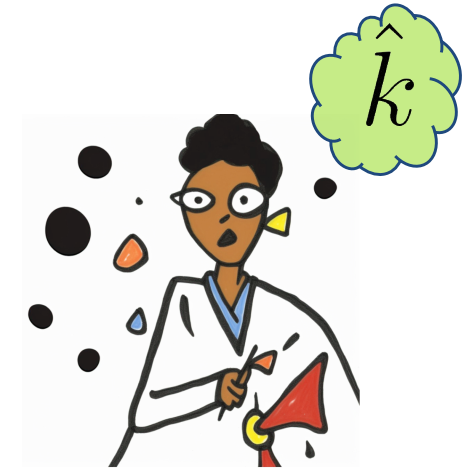
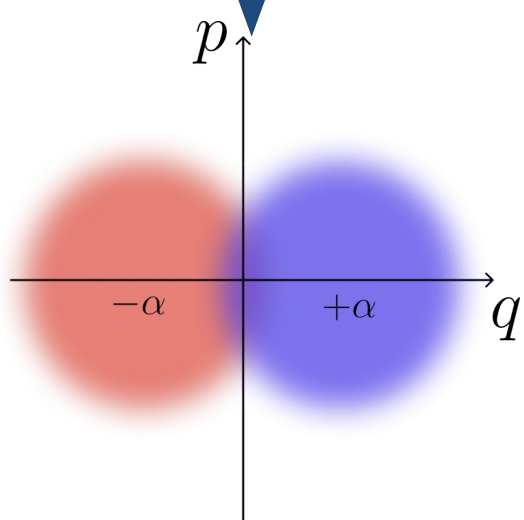
$k$

- Distinguish between  $\{|\alpha\rangle, |-\alpha\rangle\}$

$$|\alpha_k\rangle = e^{-\frac{|\alpha|^2}{2}} \sum_{n=0}^{\infty} \frac{((-1)^k \alpha)^n}{\sqrt{n!}} |n\rangle$$

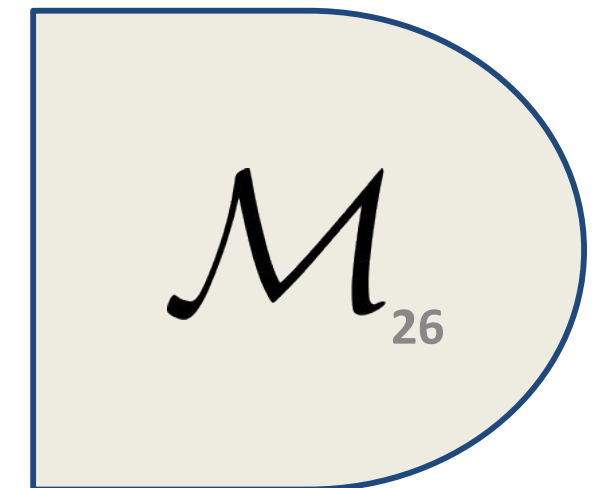
- POVM  $\mathcal{M} = \{M_0, M_1\}$

- $p(n|\alpha_k) = \text{Tr}(M_n |\alpha_k\rangle \langle \alpha_k|)$



$\hat{k}$

$$n \sim p(n|\alpha_k)$$

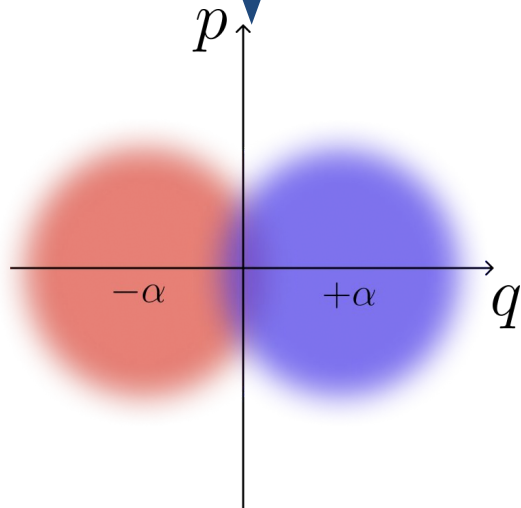


# Success probability & Helstrom bound



$k$

$$|\alpha_k\rangle = e^{-\frac{|\alpha|^2}{2}} \sum_{n=0}^{\infty} \frac{((-1)^k \alpha)^n}{\sqrt{n!}} |n\rangle$$



- **Success probability;  $g(n)$  guessing rule**

$$P_s(\mathcal{M}, g) = \sum_n p(n|\alpha_{\hat{k}}) \text{pr}(\alpha_{\hat{k}}) |_{\hat{k}=g(n)}$$

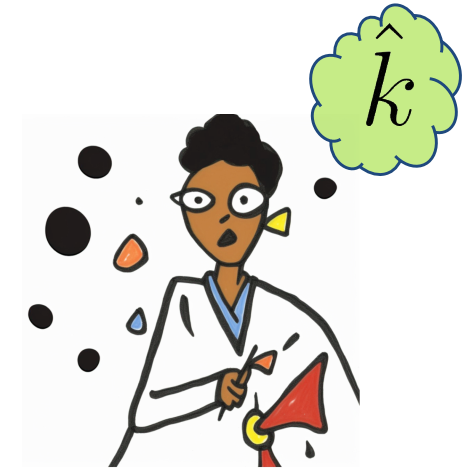
- **Helstrom bound**

$$\begin{aligned} P_s(\mathcal{M}, g) &\leq P_s(\mathcal{M}^*, g^*) \\ &= \frac{1}{2} \left( 1 + \|p_0 \rho_0 - p_1 \rho_1\|_1 \right) \end{aligned}$$

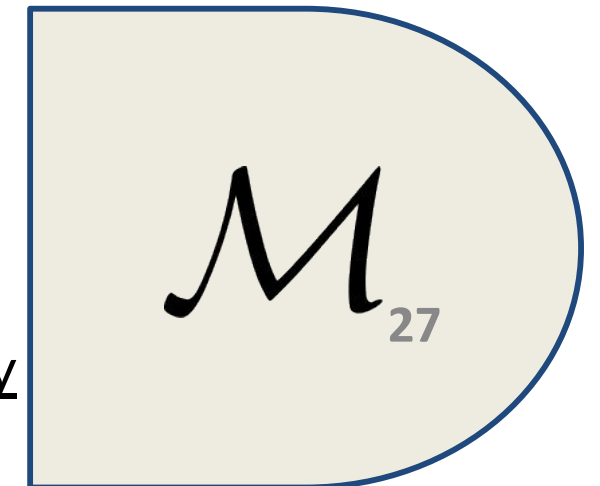
- **Optimal:**  $|\text{cat}\rangle \propto |\alpha\rangle + |-\alpha\rangle$

Can be realized w/ linear optics + on/off detectors:

- **Feed(forward) protocol on consecutive Kennedy receivers**



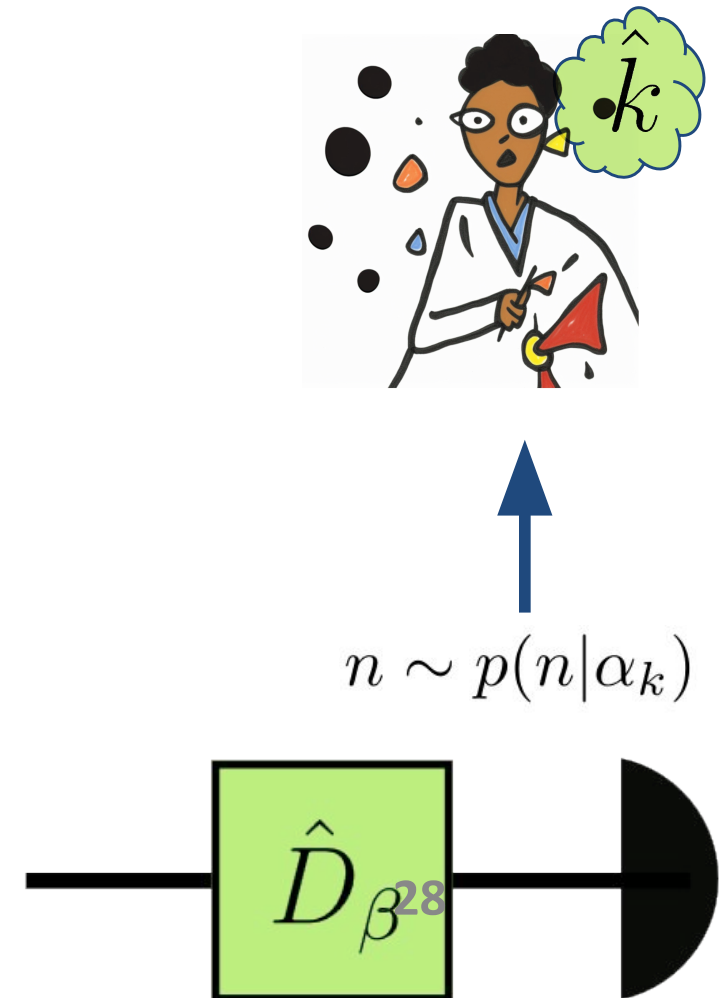
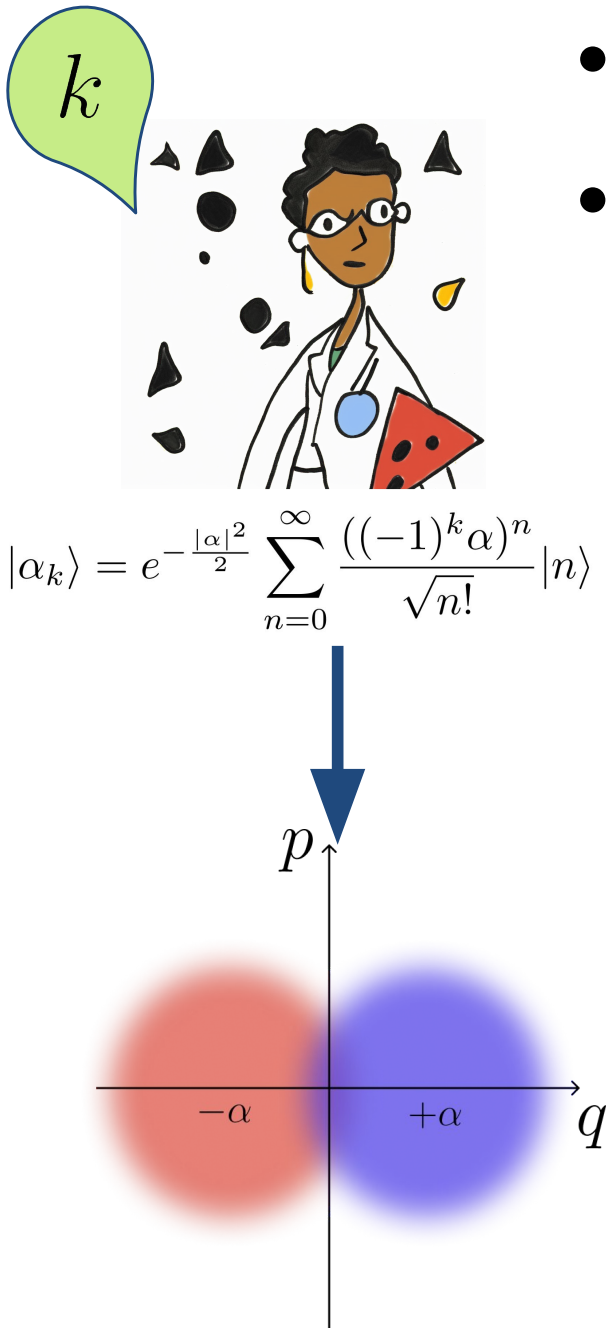
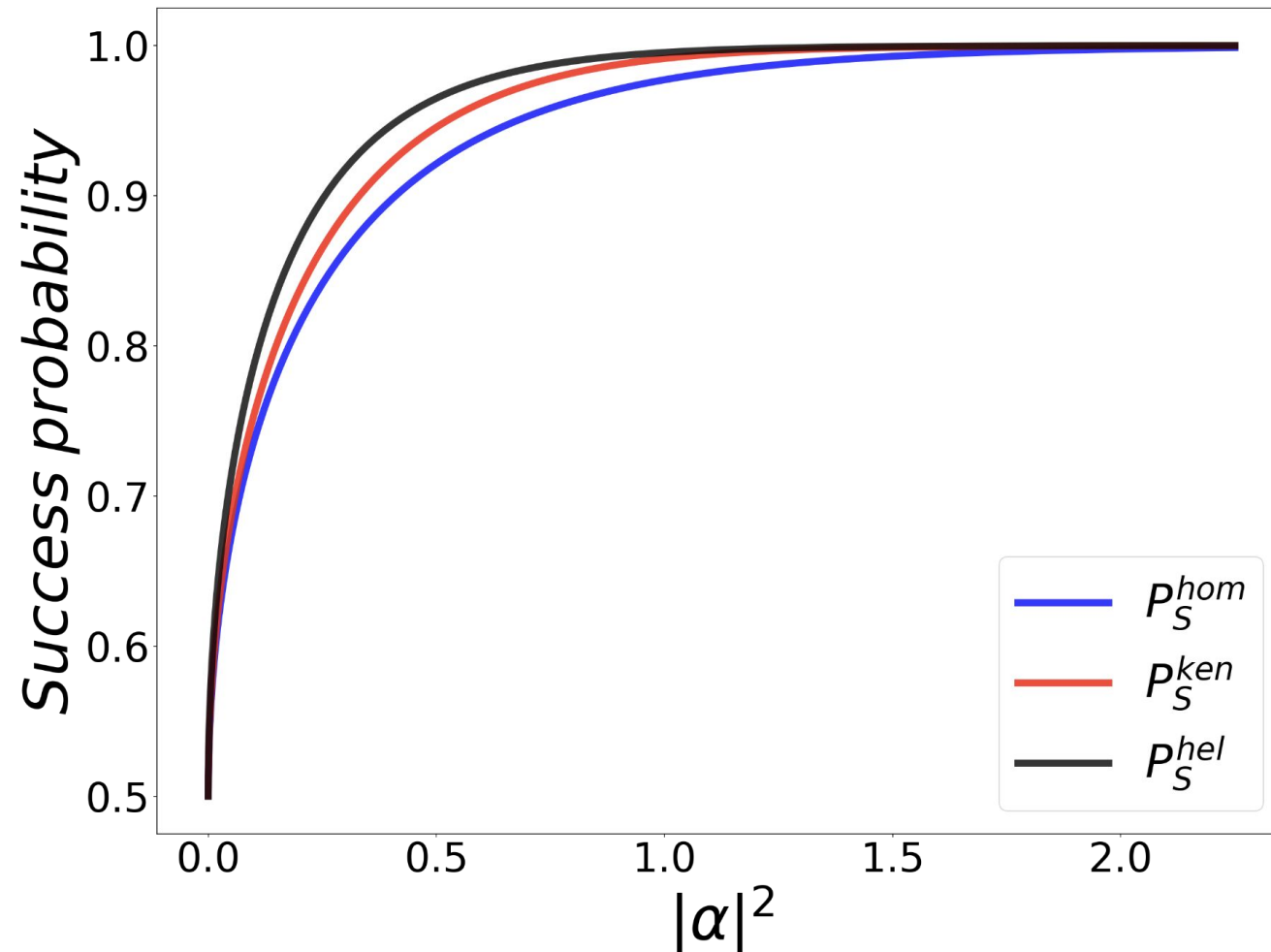
$$n \sim p(n|\alpha_k)$$



# Kennedy receiver

- Photon-detection measurement (**non-gaussian**)  $\{|0\rangle\langle 0|, I - |0\rangle\langle 0|\}$

- $$P_s^{ken} = \sum_{o=0,1} \max_k p_k p(n|\alpha^{(k)})$$

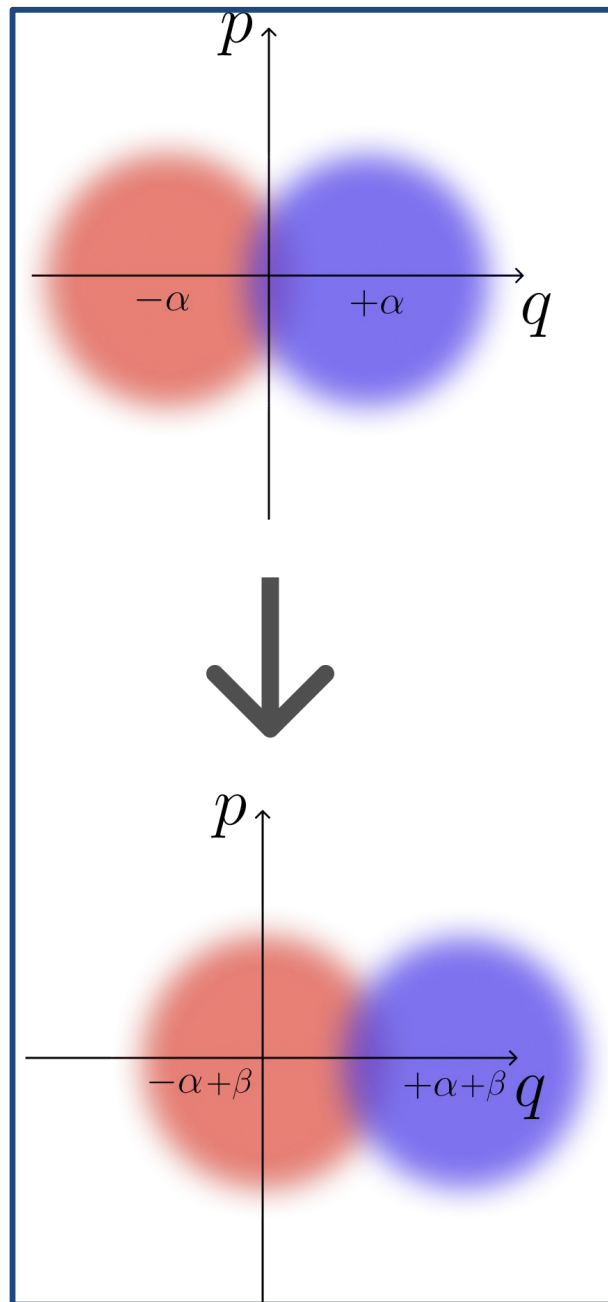
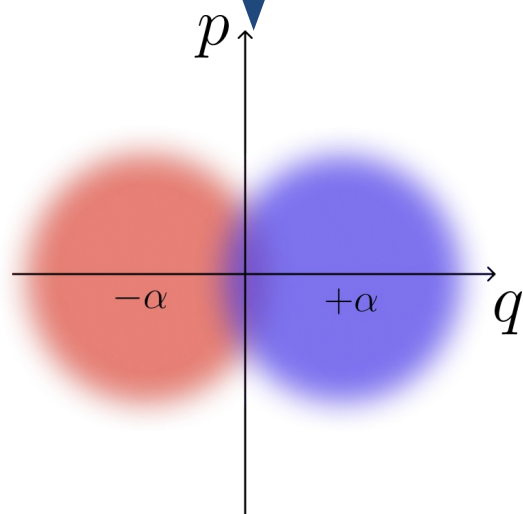


# Kennedy receiver



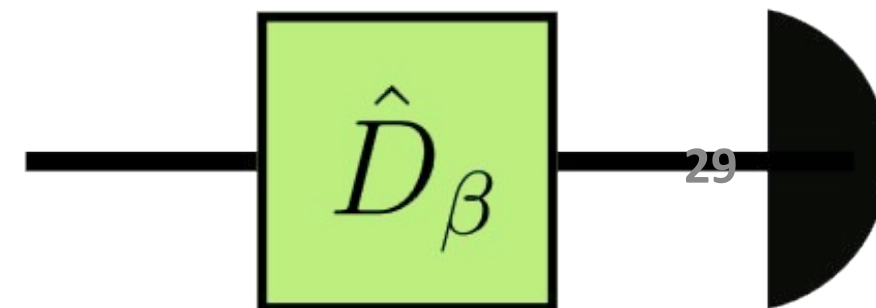
$k$

$$|\alpha_k\rangle = e^{-\frac{|\alpha|^2}{2}} \sum_{n=0}^{\infty} \frac{((-1)^k \alpha)^n}{\sqrt{n!}} |n\rangle$$



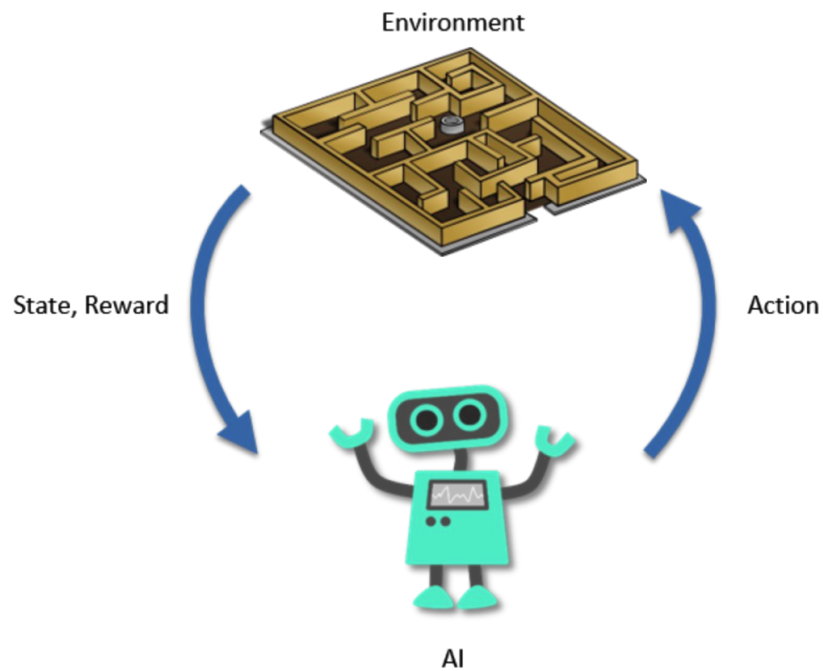
$\hat{k}$

$$n \sim p(n|\alpha_k)$$

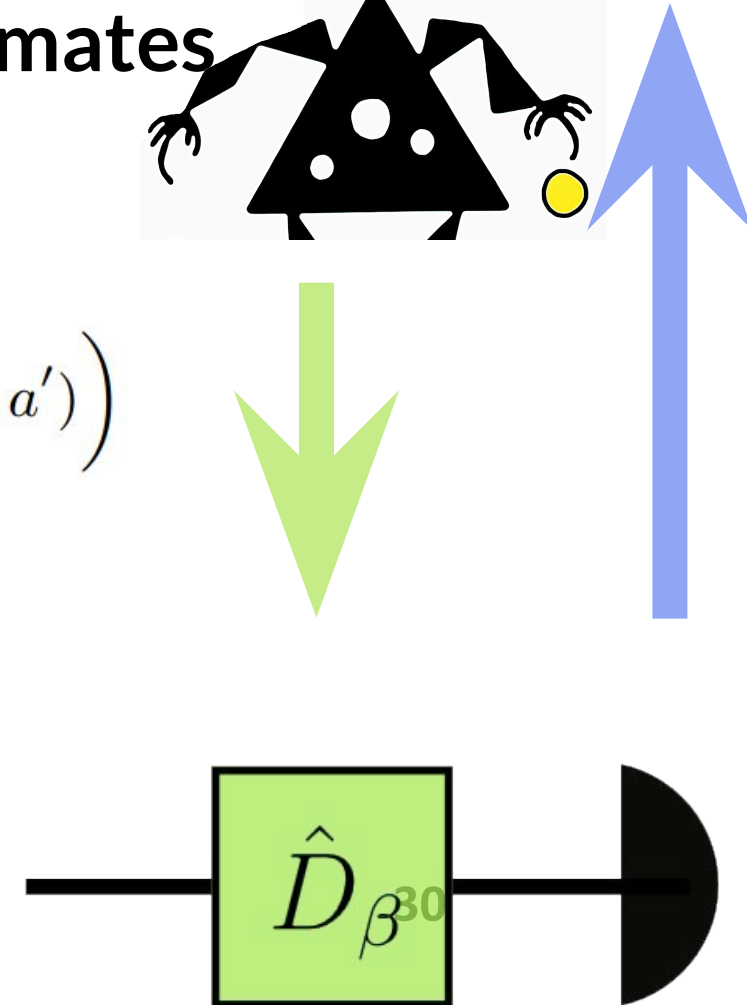
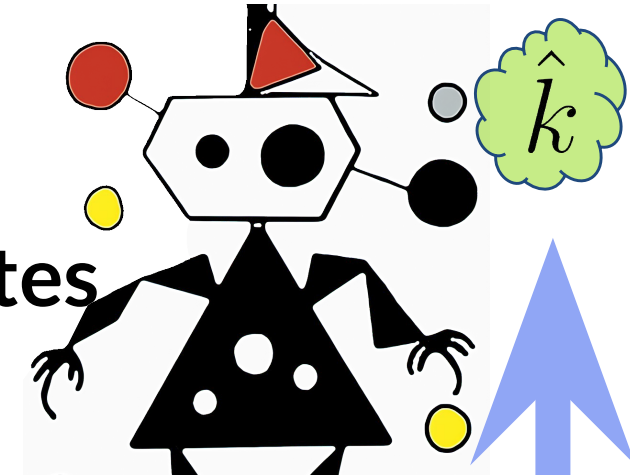


# RL calibration

- Estimate of how good a displacement is by trying it
- This is done by Q-learning, which updates these estimates

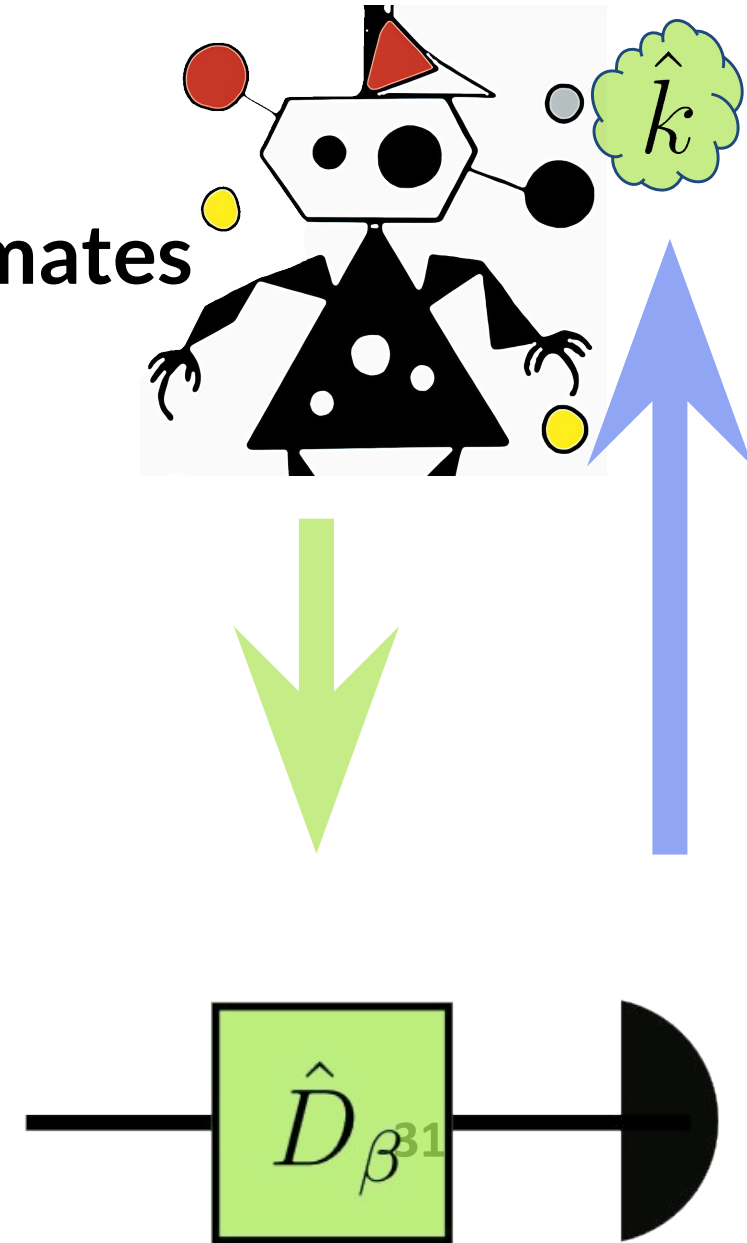
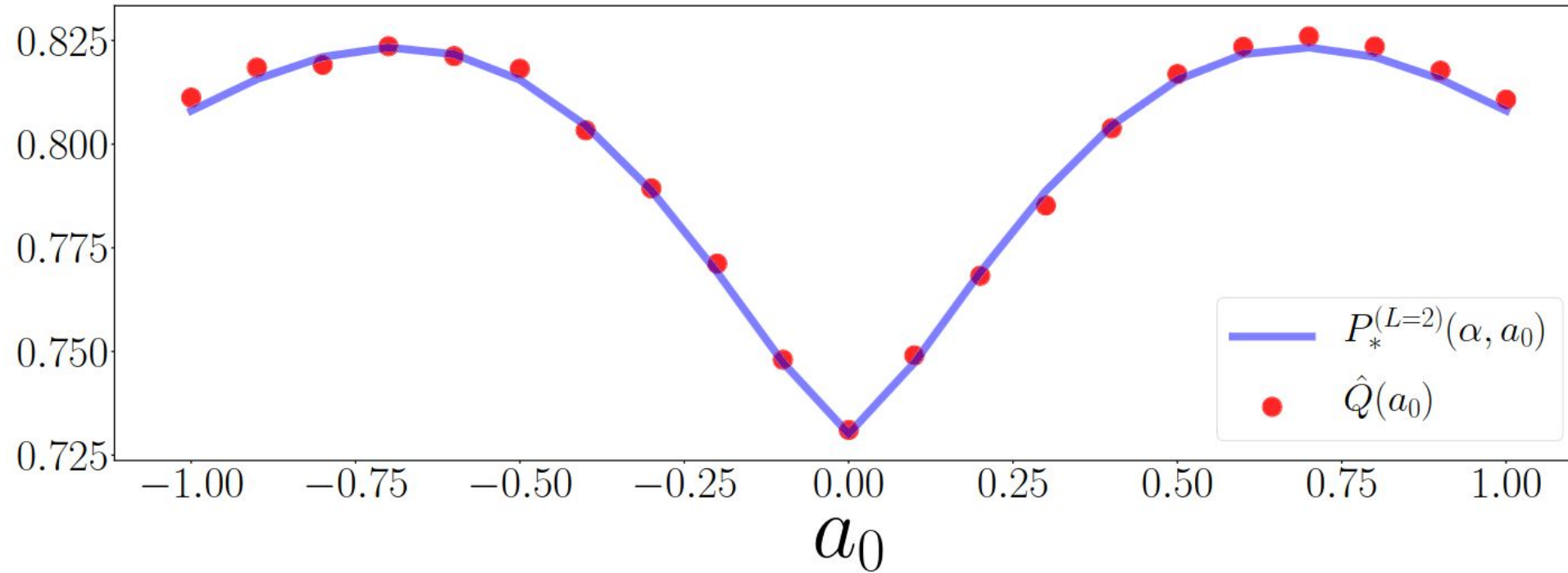


$$\hat{Q}(s_\ell, a_\ell) \leftarrow (1 - \tilde{\alpha})\hat{Q}(s_\ell, a_\ell) + \tilde{\alpha} \left( r_{\ell+1} + \gamma \max_{a' \in \mathcal{A}(s_{\ell+1})} \hat{Q}(s_{\ell+1}, a') \right)$$



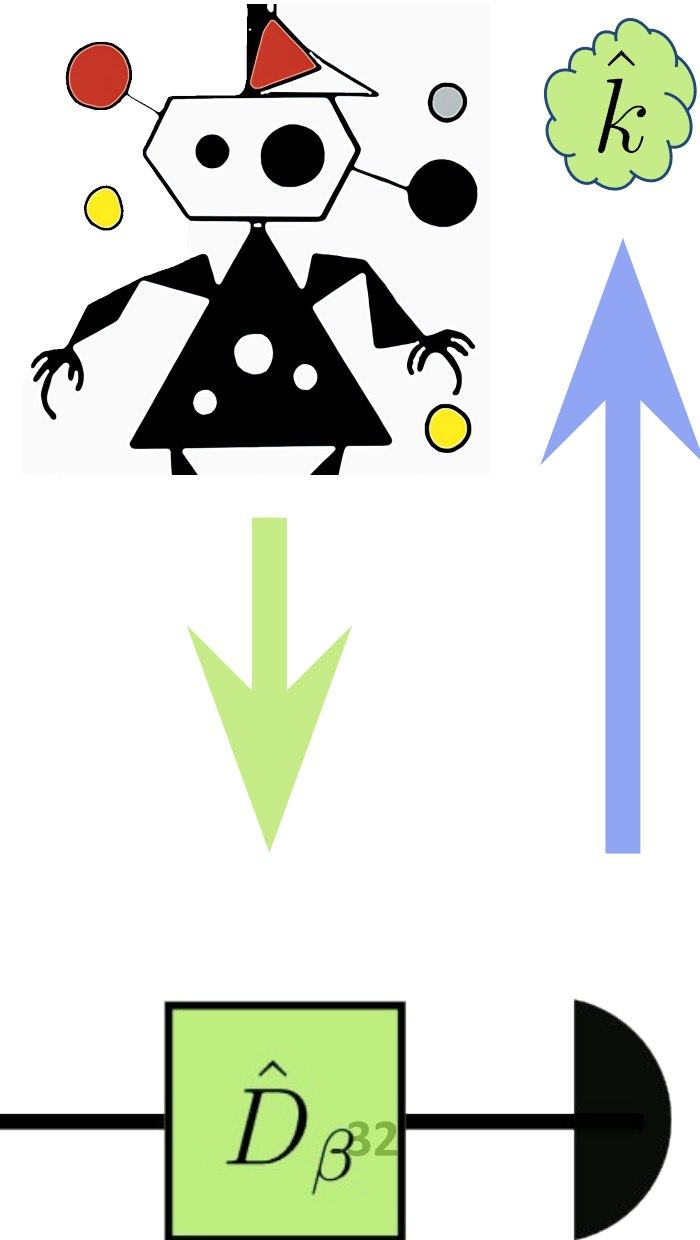
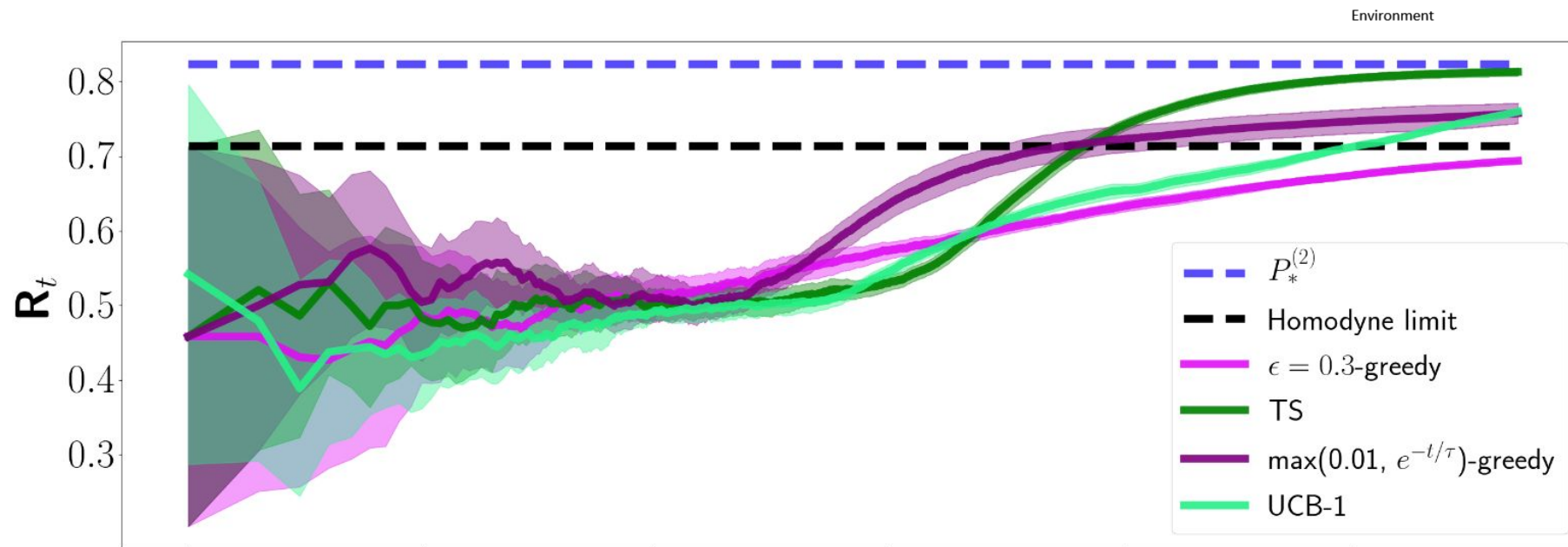
# RL calibration

- Estimate of how good a displacement is by trying it
- This is done by Q-learning, which updates these estimates



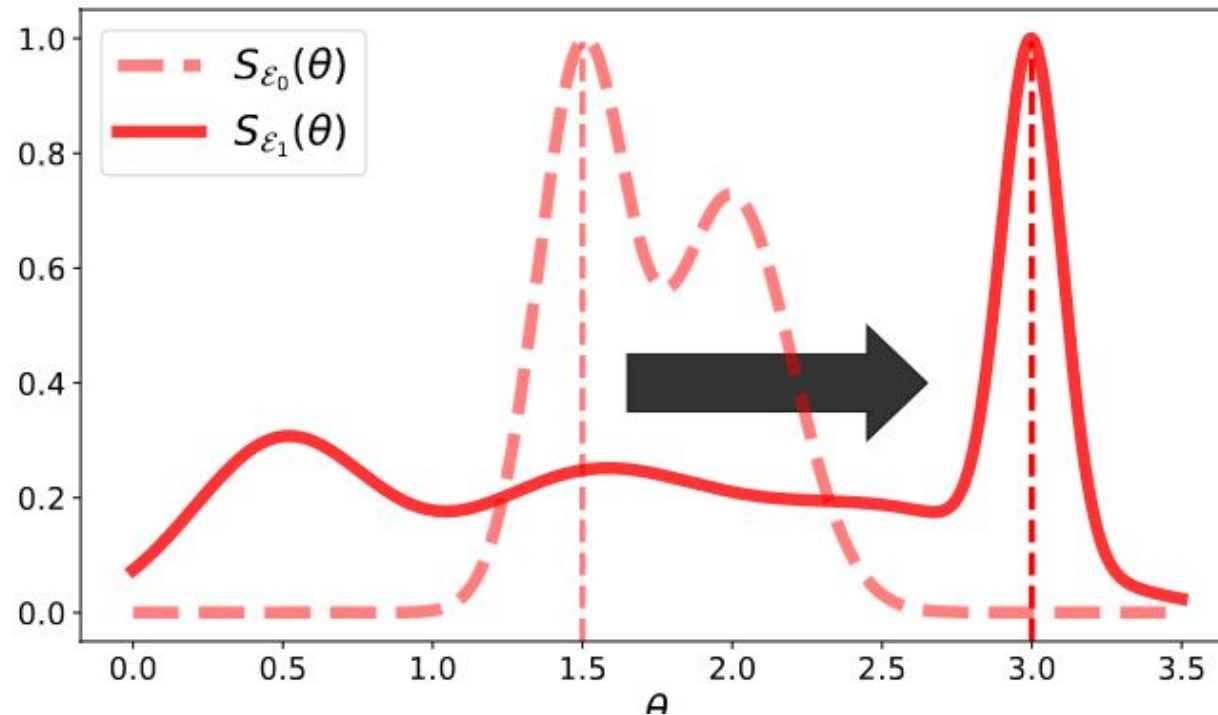
# RL calibration

- Agent guesses, reward 1/0 if ok/wrong
- Noise robustness ([PRR, 033295 (2020)])



# But if the environment changes?

$$\mathcal{E}_0 \rightarrow \mathcal{E}_1$$



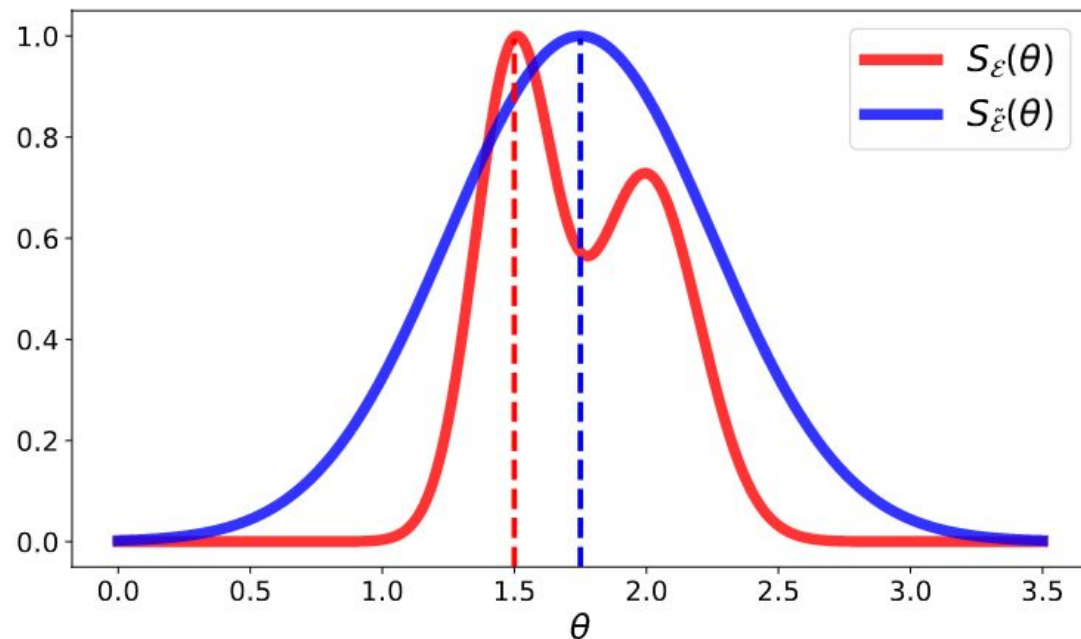
# De-calibration witness

- How to realize a change occurs?
- Monitor the reward → inaccessible when “deploying”
- Example: monitor empirical outcome probabilities

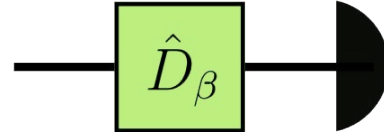


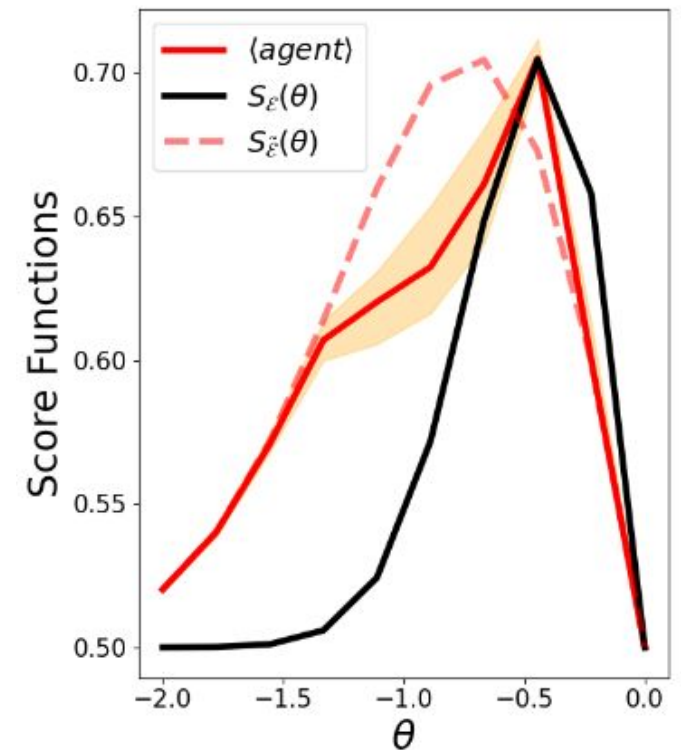
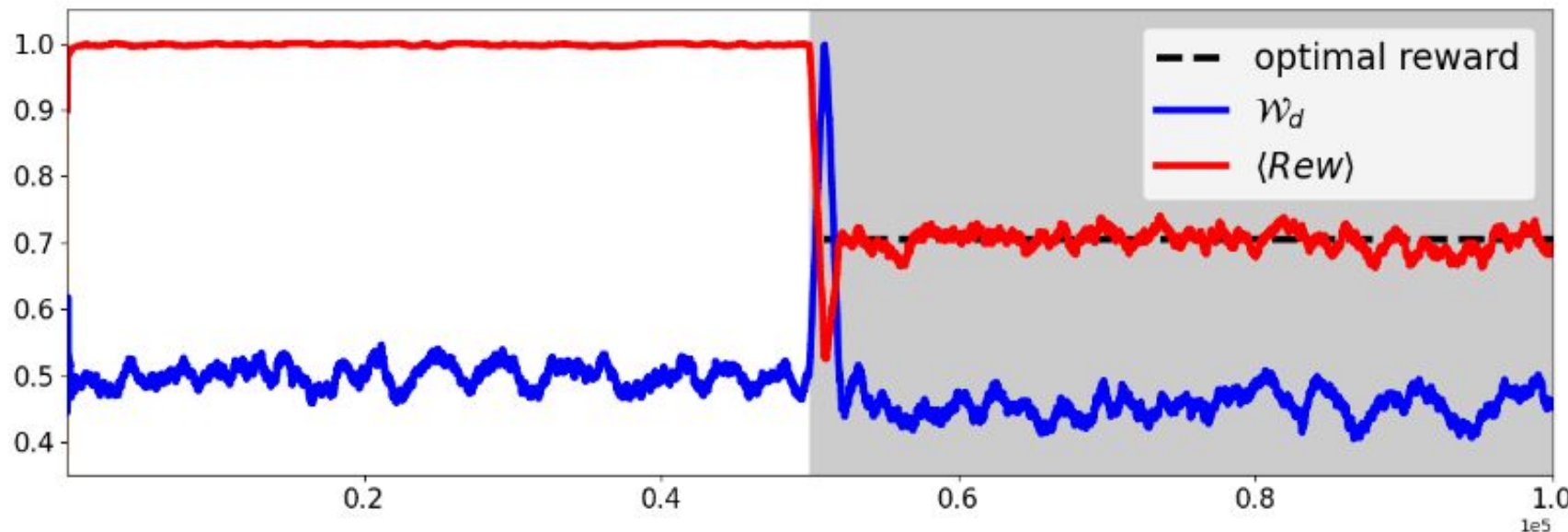
# Q-values

- Would they adapt? → Standard Q-learning yes & no (granted exploration)
- Can we do some sort of approximation?
- [Other approaches → learn map]  $\mathcal{E}_0 \rightarrow \mathcal{E}_1$
- Here: effective re-initialization of the Q-values + *fine-tune* w/ RL



# Case-study

- The displacements are faulty  $\beta \rightarrow \lambda\beta$  
- and intensity  $|\alpha|^2$  changes
- Monitor empirical outcome probabilities (decalibration witness)  $\hat{p}(n = 1)$
- Effective model: success probabilities (of un-faulty device!)



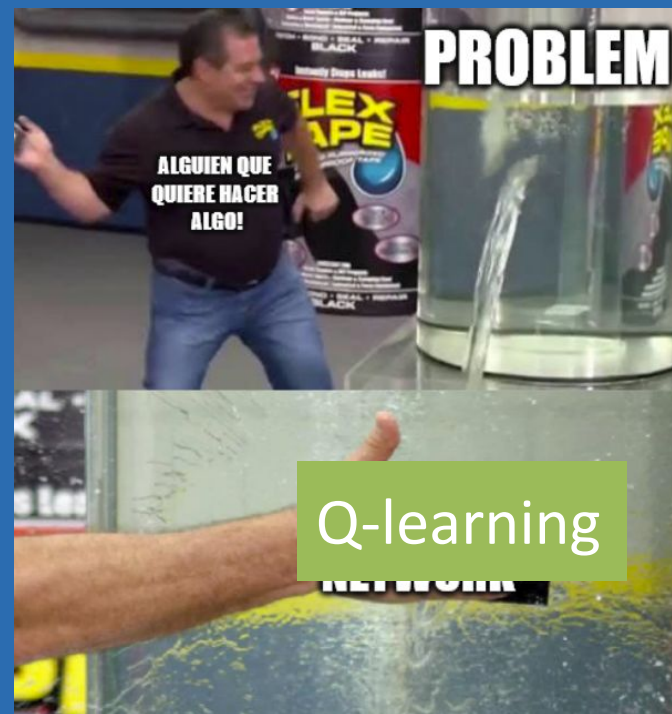
# Conclusion & questions

- We study how RL adapts to a sudden change in environment, and develop simple strategies to complement Q-learning
- We tackle a very simple case, the Kennedy receiver
- Alternative scenarios & problems to try this machinery?  
*e.g. quantum control*

I'm happy to talk :)

[mbilkis@cvc.uab.cat](mailto:mbilkis@cvc.uab.cat)

# Propuesta de tarea



# Tarea

- Leer S&B Capítulo 6
- Buscar ejemplos de aplicación de Q-learning en diferentes campos de ciencia / tech.
- *Programar y comparar SARSA & Q-learning con los ejemplos de MC-control*
- *¿¿Tres grupos que se animen a exponer ??*  
**Procedimiento, lo mismo que las clases pasada!)**

# La clase que viene

Acciones continuas

policy methods

REINFORCE